

Informe Técnico: Ethical Hacking Services (Lab 2)

1. ¿De qué trata nuestro proyecto?

La idea de "Ethical Hacking Services" nace porque los ciberataques están a la orden del día. Queremos ayudar a las empresas (sobre todo startups, fintechs y tiendas online) a descubrir sus puntos débiles antes de que los malos lo hagan. Actuamos de forma proactiva.

Buscamos que cualquier empresa tenga acceso a servicios de "Red Team" y pruebas de penetración sin importar su tamaño, mediante una plataforma en la nube rápida y segura. Todo funciona bajo un modelo de servicio (SECaaS), donde los clientes pueden agendar revisiones y ver los resultados en tiempo real con total confidencialidad.

2. ¿Cómo armamos la arquitectura?

Para que todo esto funcione bien y no se caiga, armamos la infraestructura en AWS. Nos enfocamos en que sea segura por diseño, que siempre esté disponible y que crezca sola si entra mucho tráfico. Básicamente, el flujo es el siguiente:

```
[Usuarios / Clientes]
  |
  v
[Balanceador de Carga (ALB) - Red Pública]
  |
  +---> [AWS Fargate (Web/API AZ 1) - Red Privada]
  |
  +---> [AWS Fargate (Web/API AZ 2) - Red Privada]
```

El tráfico de internet llega primero a un balanceador de carga (ALB) que está en la parte pública. Este componente se encarga de que la conexión sea segura. Luego, el balanceador manda el tráfico a nuestros contenedores, que están totalmente escondidos en subredes privadas. Así, nadie desde internet puede tocar nuestros servidores de forma directa.

3. ¿Por qué tomamos estas decisiones técnicas?

Fargate en lugar de servidores tradicionales (EC2)

Elegimos AWS Fargate para correr nuestros contenedores porque no queríamos perder tiempo administrando sistemas operativos, instalando parches o configurando servidores EC2. Además, al ser una empresa de hacking, la seguridad es vital. Fargate aísla cada contenedor; si por algún motivo logran entrar a uno, no tienen cómo saltar a un servidor principal porque simplemente no hay un servidor principal. El contenedor vive para hacer su trabajo y luego desaparece.

CPU y Memoria justos (0.5 vCPU y 1 GB RAM)

No le pusimos recursos de sobra. Nuestra app es bastante ligera porque funciona con microservicios. Dándole 1 GB de RAM estamos sobrados para que no se nos caiga por falta de memoria. Y con 0.5 vCPU es más que suficiente porque el trabajo pesado de conexión segura ya lo hace el balanceador de carga. Si nos empieza a llegar demasiada gente, simplemente encendemos más contenedores iguales en vez de hacer uno más grande y costoso.

4. Hablemos de los costos

Un buen diseño técnico no sirve de nada si nos deja en bancarrota, así que intentamos equilibrar la parte técnica con el dinero.

- **Fargate:** Solo nos cobran por segundo mientras el contenedor está prendido.
- **Balanceador de carga (ALB):** Tiene un costo fijo mensual, pero vale cada centavo porque es lo que nos da seguridad y mantiene la plataforma viva.
- **Ahorros:** Para este laboratorio y para pruebas, usamos "Fargate Spot". Básicamente, AWS nos alquila su capacidad sobrante súper barata (hasta un 70% de descuento). Como nuestra app no guarda información vital directamente en el contenedor, si AWS nos apaga un contenedor de repente, no pasa nada, se prende otro nuevo.

5. Ejecución y Despliegue

A continuación, mostramos cómo automatizamos todo este proceso usando scripts, para no tener que hacer clics manuales en la consola de AWS.

Paso 1: Construcción de la Imagen

Aquí se observa el inicio del script `deploy.sh`. El sistema comienza preparando el entorno local y utiliza Docker para construir la imagen de nuestra aplicación web (basada en Nginx). Es el primer paso para tener nuestra app empaquetada y lista para la nube.

```
yumex@DESKTOP-S5EH8KP MINGW64 ~/Desktop/proyectos/lab2-ecs/infraestructura (master)
$ bash deploy.sh

=====
INICIANDO DESPLIEGUE SERVERLESS: ETHICALHACKIN
=====
-> Eliminando versiones locales previas de ethicalhackin:1.0 (si existen)...
-> Construyendo la imagen desde la carpeta './sitio-web'...
[+] Building 2.8s (R/R) FINTSHFD                                docker:desktop-linux
=> [internal] load build definition from dockerfile              0.0s
=> => transferring dockerfile: 101B                             0.0s
-> [internal] load metadata for docker.io/library/nginx:1.25-alpine 1.4s
-> [auth] library/nginx:pull token for registry 1.docker.io     0.0s
-> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
-> [internal] load build context                                0.0s
=> => transferring context: 151B                                  0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:1.25-alpine@sha256:516475cc129da2866742567714d6c681e5eed7b9ee0b9e9c015e462b4221a00 0.1s
=> => resolve docker.io/library/nginx:1.25-alpine@sha256:516475cc129da2866742567714d6c681e5eed7b9ee0b9e9c015e462b4221a00 0.1s
-> [2/2] COPY ./usr/share/nginx/html/                          0.1s
=> exporting to image                                           1.0s
=> => exporting layers                                           0.6s
=> => exporting manifest sha256:2d5b250e8881cc28c226f550e2bf68037c23f14407985b9992be8095fc9010 0.1s
-> => exporting config sha256:b492884b3cb521ea6775204ae8336887aec0d795d5f96aad8738478e09b1a1f 0.1s
=> => exporting attestation manifest sha256:fbdcc53a6410ea2406da8b100812c5f89fe8a321ed7d726c87521hd4c6086d4 0.1s
=> => exporting manifest list sha256:868bc09027da3beb48f369d5ca6ae66160e2dd409f46e085115063c4964aad98 0.0s
=> => naming to docker.io/library/ethicalhackin:1.0             0.0s
-> => unpacking to docker.io/library/ethicalhackin:1.0          0.0s
```

Paso 2: Despliegue en AWS

Esta es la continuación del script `deploy.sh`. Se ve cómo la imagen creada se sube al registro de AWS (ECR). Luego, crea automáticamente la red (VPC), los grupos de seguridad, el balanceador de carga y, por último, lanza los contenedores en ECS Fargate. Al finalizar, nos entrega la URL pública para acceder a la plataforma web.

```
yumex@DESKTOP-S5EH8KP MINGW64 ~/Desktop/proyectos/lab2-ecs/infraestructura (master)
$ bash deploy.sh

1. Creando repositorio en ECR...
2. Autenticando Docker con AWS ECR...
Login Succeeded
3. Etiquetando y subiendo imagen...
The push refers to repository [883146828086.dkr.ecr.us-east-1.amazonaws.com/ethicalhackin]
0d0c16747d2c: Pushed
d4bca490e609: Pushed
fc21a1d387f5: Pushed
13fcfb94648: Pushed
10377cc80a3d: Pushed
287df41c149c: Pushed
e6ef242c1570: Pushed
4ahc7066143: Pushed
5406ed7b06d9: Pushed
8a3742a9529d: Pushed
1.0: digest: sha256:868bc09027da3beb48f369d5ca6ae66160e2dd409f46e085115063c4964aad98 size: 856
4. Creando VPC y red Multi-AZ...
5. Creando Security Group...
6. Creando Application Load Balancer y Target Group...
Esperando a que el ALB esté disponible (esto puede tardar unos minutos)...
7. Creando Cluster ECS y Task Definition...
8. Desplegando Servicio en Fargate...
9. Esperando a que las tareas de Fargate estén estables y registradas en el ALB (esto puede tomar 2-3 minutos)...

=====
¡DESPLIEGUE FINALIZADO EXITOSAMENTE!
Tus contenedores están arrancando. Puedes acceder a:
URL del sitio: http://ethicalhackin-alb-915404388.us-east-1.elb.amazonaws.com
=====
```

Paso 3: Limpieza de Recursos

Por último, esta captura muestra la ejecución del script `destroy.sh`. Para evitar cobros innecesarios una vez que terminamos de usar el laboratorio, este script borra todo de forma automatizada y en orden inverso: elimina los servicios, el balanceador, los repositorios y finalmente la red completa. Al terminar, la cuenta queda totalmente limpia de recursos y gastos.

```
yumax@DESKTOP-S5EH8KP MINGW64 ~/Desktop/proyectos/lab2-ecs/infraestructura (master)
● $ bash destroy.sh
=====
INICIANDO DESTRUCCIÓN DE RECURSOS: ETHICALHACKIN
=====
1. Eliminando Servicio ECS (ethicalhackin-svc)...
   Esperando 60 segundos para evitar un error de clúster...
2. Eliminando Clúster ECS (ethicalhackin-cluster)...
3. Eliminando Application Load Balancer (ethicalhackin-alb)...
   Esperando a que AWS destruya el ALB...
   Dando 30 segundos extra de margen para desconectar el Target Group...
4. Eliminando Target Group (ethicalhackin-tg)...
5. Eliminando Repositorio ECR (ethicalhackin)...
6. Buscando la VPC del laboratorio...
   VPC detectada: vpc-0a69ac83b36d9ad70
7. Eliminando Security Group...
{
  "Return": true,
  "GroupId": "sg-0bf12a69a2e745146"
}

8. Eliminando Subredes...
9. Desacoplando y eliminando Internet Gateway...
10. Eliminando Tablas de Ruteo personalizadas...
11. Eliminando la VPC principal...
    VPC eliminada.
¡LIMPIEZA COMPLETADA CON ÉXITO!
```